

Recursive Self-Improvement in Settlement Operating Systems: Deterministic Routing, Solvency Invariants, and Mesh-Level Propagation in JasperOS, Squirrel OS, and the Universal Bridge Architecture

Author: Leon Calvin Long II

Date: August 2026

Repository: Zenodo — Open Access Research Archive

DOI: [Zenodo DOI pending assignment]

© 2026 Leon Licensed under Creative Commons Attribution 4.0 International (CC BY 4.0).

You are free to share and adapt this material with appropriate credit.

Abstract

This paper presents a formal description of a recursive self-improvement (RSI) architecture deployed across a family of settlement operating systems — JasperOS, Squirrel OS, and the Universal Bridge — designed to operate at the intersection of mesh networking, deterministic economic routing, and autonomous system optimization. Unlike classical RSI proposals grounded in unconstrained machine learning, the architecture described here achieves self-optimization through a closed-loop system of rail scoring, solvency invariants, provenance anchoring, lifecycle proofs, and mesh-level state propagation. Each optimization cycle is bounded by hard mathematical constraints that preserve system solvency and provenance integrity, ensuring that no self-modification can produce an insolvent or unprovable system state. The result is a system capable of continuous structural improvement while remaining formally verifiable at every layer. The architecture is

shown to satisfy three core AI safety properties — corrigibility, containment, and auditability — by construction, without relying on learned heuristics or probabilistic safety guarantees. This paper describes the theoretical foundations, component interactions, invariant definitions, and propagation mechanics that collectively constitute this architecture. The work is positioned within the tradition of formal methods for distributed systems and contributes a novel instantiation of bounded RSI applicable to settlement, finality, and economic routing systems operating at mesh scale.

Keywords:

Recursive self-improvement, settlement operating systems, deterministic routing, rail scoring, solvency invariants, provenance anchoring, lifecycle proofs, mesh propagation, JasperOS, Squirrel OS, Universal Bridge, distributed systems, autonomous optimization, formal verification

Table of Contents

1. Introduction

- 1.1 Motivation and Context
- 1.2 Scope and Contributions
- 1.3 Document Organization

2. Background and Related Work

- 2.1 Classical Recursive Self-Improvement
- 2.2 Settlement Operating Systems
- 2.3 Mesh Networking and Distributed State
- 2.4 Formal Verification in Autonomous Systems

3. System Overview: JasperOS, Squirrel OS, and Universal Bridge

- 3.1 JasperOS | 3.2 Squirrel OS | 3.3 Universal Bridge | 3.4 Component Interaction

4. Deterministic Routing

4.1 Definition and Formal Properties | 4.2 Rail Architecture | 4.3 Routing Table Structure
| 4.4 Self-Optimization

5. Rail Scoring

5.1 Scoring Objectives | 5.2 Scoring Function | 5.3 Score Propagation | 5.4 Feedback
Loop

6. Solvency Invariants

6.1 Definition | 6.2 Invariant Stack | 6.3 Enforcement Mechanism | 6.4 Recovery

7. Provenance Anchoring

7.1 Purpose | 7.2 Anchoring Mechanism | 7.3 Cross-System Provenance | 7.4
Integration with RSI

8. Lifecycle Proofs

8.1 Definition | 8.2 Proof Structure | 8.3 Use in RSI Cycles | 8.4 Verification

9. Mesh-Level Propagation

9.1 Architecture | 9.2 Protocol | 9.3 RSI Propagation | 9.4 Convergence Properties

10. The RSI Feedback Loop: Unified Description

10.1 The Five-Layer Stack | 10.2 Cycle Operation | 10.3 Self-Improvement Guarantee |
10.4 AI Safety Principles

11. Patent Coverage and Intellectual Property

12. Conclusion

References

1. Introduction

1.1 Motivation and Context

Settlement operating systems represent a qualitatively distinct class of computing infrastructure from conventional general-purpose operating systems. Where traditional operating systems govern the allocation of computational resources — processor time, memory, storage, and I/O — settlement operating systems govern the allocation and movement of economic, informational, and physical resources across distributed networks of participating nodes. The distinction is not merely functional but

architectural: a settlement OS must reason about finality, solvency, provenance, and obligation in addition to the classical concerns of scheduling, memory management, and fault isolation.

Classical OS design is fundamentally static with respect to its own structure. Once a kernel is deployed, its routing logic, scheduling policies, and resource allocation algorithms remain fixed until an operator-initiated update is applied. This model is adequate for environments where the computational workload is well-characterized and bounded. Settlement environments, by contrast, are inherently dynamic: routing tables must reflect continuously shifting node capabilities, resource valuations fluctuate with economic conditions, and the topology of the participating mesh changes as nodes join, depart, or degrade. A settlement OS that cannot adapt its own internal structure in response to these dynamics will either over-provision — maintaining capacity for worst-case conditions at all times — or under-serve — failing to honor committed settlements during peak periods.

This observation motivates the introduction of recursive self-improvement (RSI) into the settlement OS context. However, the term RSI carries significant theoretical and safety baggage from the AI literature, where it typically refers to a system capable of rewriting its own cognitive architecture without formal bounds. The architecture described in this paper is explicitly not of that kind. Rather, RSI is instantiated here as a formally bounded, invariant-preserving optimization loop: the system continuously scores its own routing rails, identifies inefficiencies, proposes structural improvements, checks those improvements against a stack of hard mathematical invariants, and propagates accepted improvements across the mesh. No self-modification is permitted that would violate solvency, provenance integrity, or lifecycle correctness. The result is a system that improves continuously while remaining formally verifiable at every layer.

This architecture emerged from patented methods for deterministic routing and solvency management in multi-party settlement networks, developed by the author and described herein for the purposes of scientific and scholarly communication.

1.2 Scope and Contributions

This paper makes the following contributions:

1. A formal definition of recursive self-improvement as applied to settlement operating systems, distinguishing this instantiation from classical unbounded RSI.
2. A complete description of the five-layer invariant stack — rail scoring, solvency invariants, provenance anchoring, lifecycle proofs, and mesh propagation — that bounds and structures each optimization cycle.
3. An architectural description of how three distinct system components — JasperOS, Squirrel OS, and the Universal Bridge — instantiate these layers at different tiers of the deployment hierarchy.
4. A unified description of the RSI feedback loop, including a step-by-step cycle walkthrough and an informal convergence proof for the mesh propagation protocol.
5. A discussion of how the architecture satisfies corrigibility, containment, and auditability — three foundational properties of AI safety research — by formal construction.

1.3 Document Organization

The remainder of this paper is organized as follows. Section 2 reviews relevant background literature across four domains: classical RSI theory, settlement operating systems, mesh networking and distributed state, and formal verification methods. Section 3 provides a system-level overview of JasperOS, Squirrel OS, and the Universal Bridge, including a component

comparison table. Sections 4 through 9 describe each of the five architectural layers in detail. Section 10 presents a unified description of the RSI feedback loop. Section 11 addresses patent coverage and intellectual property. Section 12 concludes with a summary and directions for future work. References follow.

2. Background and Related Work

2.1 Classical Recursive Self-Improvement

The theoretical foundations of recursive self-improvement trace to Gödelian self-reference: a sufficiently powerful formal system can represent statements about its own behavior, raising the possibility of a system that can reason about, evaluate, and modify its own computational structure. Schmidhuber's Gödel Machines formalize this intuition, positing an agent that rewrites any part of its own software if and only if it can prove, using a formal axiomatic system, that the rewrite will improve expected future reward. The appeal of this approach is its theoretical elegance; the limitation is its practical intractability and its dependence on an unbounded proof search.

AIXI-style self-modification proposals extend this lineage, describing agents that can in principle model and modify any computable environment, including themselves. Again, the theoretical guarantees are strong but the computational requirements are unphysical, and crucially, neither Gödel Machines nor AIXI-style agents carry inherent solvency or provenance constraints. The architecture described in this paper diverges fundamentally: improvement is bounded by invariants rather than driven by unbounded proof search, and the optimization objective is not a reward function over arbitrary futures but a concrete, measurable score across a defined set of rail efficiency metrics.

2.2 Settlement Operating Systems

A settlement operating system, as used in this paper, denotes a class of distributed systems whose primary function is the management of economic finality across a network of participating nodes. The term "finality" here carries its meaning from payment systems and distributed ledger theory: a settlement is final when it is irrevocable, fully honored, and provably recorded. Key properties that distinguish a settlement OS from a general-purpose distributed system include: atomicity of settlement (all-or-nothing commitment semantics), deterministic routing (every routing decision must be reproducible given the same input state), provenance traceability (every value movement must be traceable to its origin), and solvency maintenance (the system must never commit to more than it can honor).

Prior work in payment network infrastructure, including the ISO 20022 messaging standards and various central bank real-time gross settlement (RTGS) architectures, addresses subsets of these requirements. The architecture described here generalizes beyond payment-specific domains to cover arbitrary resource flows — economic, informational, and physical — within a unified OS framework.

2.3 Mesh Networking and Distributed State

Mesh topologies provide the substrate for the distributed optimization described in this paper. In a mesh network, each node maintains direct communication channels with one or more peer nodes, and information propagates through the network via multi-hop relay rather than through a centralized hub. This architecture provides resilience — no single node failure disconnects the network — at the cost of coordination complexity.

The gossip protocol literature provides well-studied mechanisms for eventual consistency in mesh networks: nodes periodically exchange state with random peers, and convergence to a consistent global state is guaranteed under weak connectivity assumptions. Byzantine fault tolerance (BFT) work,

beginning with Lamport, Shostak, and Pease's foundational 1982 paper, establishes the conditions under which a distributed system can reach agreement in the presence of malicious or faulty nodes. Distributed ledger systems, including Nakamoto's Bitcoin and subsequent blockchain architectures, apply BFT principles to the specific problem of maintaining an append-only log of finalized state transitions.

The architecture described here diverges from the pure consistency-maintenance paradigm of these prior works: the goal is not merely to propagate a consistent state but to propagate structurally optimized state — routing tables and rail configurations that have been formally checked against solvency and provenance invariants before propagation.

2.4 Formal Verification in Autonomous Systems

The formal methods community has produced a rich toolkit for reasoning about the correctness of distributed systems. TLA+ (Temporal Logic of Actions), developed by Leslie Lamport, provides a specification language for describing and model-checking concurrent and distributed algorithms. The SPIN model checker, described by Holzmann, provides automated verification of protocol specifications against linear temporal logic properties. Coq-based proof assistants enable machine-checked proofs of system invariants for implementations extracted from verified specifications.

This architecture is positioned within this formal verification tradition. The five invariants described in Section 6 are amenable to TLA+ specification; a complete TLA+ model of the invariant stack is identified as future work (Section 12.2). The provenance anchoring and lifecycle proof mechanisms described in Sections 7 and 8 can be understood as runtime witnesses for invariant preservation, providing audit evidence that formal verification guarantees hold in the deployed system.

3. System Overview: JasperOS, Squirrel OS, and Universal Bridge

3.1 JasperOS

JasperOS is the primary settlement operating system in this architecture, responsible for managing high-throughput deterministic routing across economic nodes at the root tier of the settlement stack. Its core capabilities encompass three domains: rail management (the creation, configuration, scoring, and deprecation of routing rails), the scoring engine (the aggregation and canonicalization of rail scores from across the mesh), and the solvency ledger (the authoritative record of all committed obligations, available capacities, and solvency headroom across all rails and nodes).

In the deployment hierarchy, JasperOS occupies the root-level position. All other OS instances — whether Squirrel OS leaf nodes or Universal Bridge boundary nodes — coordinate with JasperOS to obtain canonical system state. JasperOS is the ultimate arbiter of routing table versions, rail scores, lifecycle proof logs, and provenance chains. However, JasperOS does not dictate local decisions at leaf nodes in real time; rather, it publishes signed optimization updates that leaf nodes apply autonomously after local invariant verification.

3.2 Squirrel OS

Squirrel OS is a lightweight, edge-deployed operating system designed for resource-constrained nodes at the leaf tier of the mesh. Its design philosophy prioritizes minimal resource consumption while maintaining the full invariant-checking and provenance-anchoring capabilities required for participation in the RSI feedback loop. Core capabilities include local rail scoring (computing score approximations based on locally observable rail metrics), lightweight provenance anchoring (maintaining a local subset of the provenance DAG sufficient for local verification), and upstream state

propagation (signing and forwarding local scores and lifecycle events to JasperOS via the mesh).

Squirrel OS nodes do not hold canonical system state. Their routing tables and rail scores are local approximations of the canonical state maintained by JasperOS, updated as optimization messages arrive via the propagation protocol. This distinction between canonical state (JasperOS) and local approximation (Squirrel OS) is fundamental to the scalability of the architecture: leaf nodes can operate independently during network partitions, applying locally valid optimizations, while the system converges to canonical state once connectivity is restored.

3.3 Universal Bridge

The Universal Bridge is the interoperability and translation layer that mediates all interaction between the settlement mesh and external networks, systems, or counterparties. Its core capabilities address the fundamental challenge of heterogeneity: external systems may use different routing protocols, provenance formats, lifecycle models, and economic representations than the internal settlement mesh. The Universal Bridge provides cross-protocol deterministic routing (translating external routing requests into internally deterministic form), provenance normalization (mapping external provenance formats to the internal DAG structure), and lifecycle proof bridging (generating internal lifecycle proofs for externally-originated events).

The Universal Bridge occupies a boundary position in the deployment hierarchy: it sits between the settlement mesh and all external systems, and every external ingress or egress event must transit it. This position makes it the natural enforcement point for cross-system solvency invariants (Invariant I_5 , described in Section 6.2). The Universal Bridge rejects any external request that would cause the internal mesh to violate any invariant in the stack.

3.4 Component Interaction

The three system components interact in a layered topology. JasperOS serves as the root node, connected to all Squirrel OS leaf nodes via bidirectional mesh propagation channels. The Universal Bridge connects to JasperOS and to external networks simultaneously, translating all cross-boundary traffic. Optimization cycles originate at Squirrel OS leaf nodes, which compute local rail scores and propagate them to JasperOS. JasperOS aggregates scores, generates optimization proposals, verifies them against the full invariant stack, and propagates accepted optimizations back to the affected leaf nodes. The Universal Bridge participates in this loop only when external-facing rails are affected by an optimization cycle.

The following table compares the three system components across five architectural dimensions:

Dimension	JasperOS	Squirrel OS	Universal Bridge
Deployment Tier	Root (canonical authority)	Leaf (edge, resource-constrained)	Boundary (mesh-to-external interface)
State Authority	Holds canonical state for all rails, scores, proofs, and provenance chains	Holds local approximations; defers to JasperOS for canonical resolution	Holds canonical cross-system mapping tables and boundary invariant state
Scoring Scope	Global aggregate scoring across all rails and all nodes	Local scoring for directly observable rails only	Cross-boundary scoring for external-facing rails
Propagation Role	Publishes canonical optimizations; receives score	Sends local scores upstream; receives and	Normalizes and forwards cross-system events; enforces

Dimension	JasperOS	Squirrel OS	Universal Bridge
	updates from leaves	applies canonical optimizations	boundary invariants
External Interface	None (internal only; external access via Universal Bridge)	None (internal only)	Primary external interface for all ingress and egress

4. Deterministic Routing

4.1 Definition and Formal Properties

Deterministic routing is defined as follows: a routing function R is deterministic if and only if, given an identical system state S and an identical input I , it always produces an identical output routing decision O . Formally:

$R(S, I) = O$ where R is a pure function with no stochastic components.

This definition has three immediate corollaries that are essential in settlement contexts. First, auditability: any routing decision can be replayed from its inputs and will produce the same output, allowing external auditors to verify that routing decisions were correctly computed. Second, proof generation: because routing decisions are deterministic, they can serve as inputs to lifecycle proofs (Section 8) without introducing non-deterministic dependencies that would prevent proof verification. Third, consistency: all nodes that hold the same system state and receive the same input will compute the same routing decision, eliminating divergence due to stochastic routing choices.

Determinism requires that the system state S used in routing computations be consistent across all nodes that participate in the same routing decision.

This consistency requirement motivates the signed, hash-chained routing table protocol described in Section 4.3.

4.2 Rail Architecture

Rails are the fundamental routing primitive of the settlement OS architecture. A rail is defined as a typed, directed routing channel between two nodes, carrying a specific category of resource flow. Three rail types are defined:

6. **Economic Rails:** Carry value transfer payloads — commitments, settlements, and economic obligations between nodes.
7. **Informational Rails:** Carry metadata, state updates, scoring messages, provenance records, and lifecycle proofs.
8. **Control Rails:** Carry system commands, optimization updates, routing table versions, and invariant enforcement signals.

Rail selection for a given routing decision is determined by a scored selection algorithm that evaluates each candidate rail across four dimensions: current capacity utilization (lower is preferred), observed end-to-end latency (lower is preferred), solvency headroom (higher is preferred, reflecting how far the rail is from its capacity limit), and provenance status (rails with incomplete or flagged provenance chains are penalized). The rail with the highest composite score is selected, subject to the constraint that the selected rail must pass a solvency invariant check before the routing decision is committed.

4.3 Routing Table Structure and Update Protocol

Routing tables in this architecture are versioned, cryptographically signed, and hash-chained. Each table version carries: a monotonically increasing version number, the signing node's identity and public key, a hash of the previous table version (enabling chain verification), the set of current rail configurations and their associated scores, and a timestamp of generation.

Routing table updates follow a strict protocol: a candidate update is first constructed by JasperOS based on the current aggregate score state; the candidate is then checked against the full solvency invariant stack (I_1 through I_5); if the check passes, the update is signed, hash-chained to the current table version, and propagated to all affected nodes via the mesh propagation protocol; if the check fails, the candidate is rejected, a cause code is recorded in the provenance log, and an alternative candidate is generated. No routing table update may be applied by any node until the update's invariant compliance has been locally verified by that node.

4.4 Self-Optimization of Routing

The routing self-optimization loop is the first and most fundamental RSI cycle in the architecture. Rail scores, as described in Section 5, are computed continuously and feed back into routing table update decisions. Specifically, when the aggregate score delta — the difference between the current aggregate score and the score of the proposed new routing configuration — exceeds a configurable threshold $\Delta_{\min}$, a routing table re-optimization is triggered.

Re-optimization is constrained to produce only routing configurations that preserve all solvency invariants. The optimization search is therefore not unconstrained: it operates over the feasible region defined by I_1 through I_5 . Within this region, the optimization selects the configuration that maximizes the aggregate composite rail score $S(r)$ as defined in Section 5.2. This bounded optimization is the formal sense in which routing is "self-improving": each triggered re-optimization produces a routing configuration with higher expected aggregate score than the configuration it replaces, within the invariant-bounded feasible region.

5. Rail Scoring

5.1 Scoring Objectives

Rail scoring serves as the primary signal-generation mechanism of the RSI feedback loop. A score quantifies the current efficiency, reliability, and solvency health of a given rail, providing the information necessary for the routing optimizer to identify underperforming rails and propose improvements. The scoring framework is deliberately multi-objective: optimizing for a single metric such as throughput would systematically sacrifice latency, solvency headroom, or provenance integrity — all of which are essential safety properties in a settlement context.

Four primary scoring dimensions are defined: throughput (the rate at which the rail successfully delivers payloads, normalized to its rated capacity), latency (the end-to-end delivery time for payloads, inverted so that lower latency produces a higher score), solvency headroom (the margin between the rail's committed load and its maximum capacity), and provenance integrity (a binary-like measure reflecting whether the rail's provenance chain is complete, unbroken, and all records are validly signed).

5.2 Scoring Function

The composite rail score for a rail r is defined as a weighted linear combination of the four dimension scores:

$$S(r) = w_1 \cdot T(r) + w_2 \cdot L(r) + w_3 \cdot H(r) + w_4 \cdot P(r)$$

where $T(r)$ denotes the throughput score, $L(r)$ denotes the latency score (inverted), $H(r)$ denotes the solvency headroom score, and $P(r)$ denotes the provenance integrity score. Each component score is normalized to the interval $[0, 1]$. The weights w_1 through w_4 are system-tunable parameters subject to the constraint $w_1 + w_2 + w_3 + w_4 = 1$, ensuring that the composite score remains in $[0, 1]$.

Notably, the weights w_1 through w_4 are themselves subject to RSI optimization: the system can adjust the relative importance of scoring

dimensions in response to observed system conditions. However, weight adjustments are constrained: the solvency headroom weight w_3 may never be set below a configurable minimum $w_{3\min} > 0$, ensuring that solvency is always a positive contributor to the routing objective. This constraint is a meta-level solvency invariant that applies to the scoring function itself.

Score computation occurs at configurable intervals. JasperOS holds and publishes canonical scores for all rails in the mesh. Squirrel OS nodes hold local approximations computed from locally observable metrics; these approximations are propagated to JasperOS at each score update interval for aggregation into canonical scores.

5.3 Score Propagation

Local scores computed by Squirrel OS nodes are propagated to JasperOS via signed score messages carried on informational rails. Each score message contains: the originating node identity, the rail identifier, the locally computed scores for each dimension, the composite score, the timestamp of computation, and a cryptographic signature. JasperOS validates the signature before incorporating the score into the global aggregation.

JasperOS aggregates received local scores using a weighted mesh average, where the weights reflect each reporting node's position in the mesh and the recency of its reported scores. The aggregated canonical score for each rail is published to all nodes in the mesh at a configurable broadcast interval. Score history is retained by JasperOS in a hash-chained log that serves as an input to lifecycle proof generation (Section 8): the historical score trajectory of a rail provides evidence of its lifecycle stage transitions.

5.4 Feedback Loop

The scoring feedback loop operates as follows. Score updates arrive at JasperOS from Squirrel OS nodes continuously. JasperOS computes score deltas — differences between the current canonical score and the newly

aggregated score for each rail. If a score delta exceeds the threshold $\Delta_{\min}$, a routing table re-evaluation is triggered as described in Section 4.4. This re-evaluation constitutes the first layer of the RSI feedback loop: scores drive routing changes, which alter load distributions across rails, which in turn alter future scores.

In addition to triggering routing re-evaluation, extreme score degradation — specifically, cases where the solvency headroom score $H(r)$ falls below a configurable threshold $H_{\min}$ — triggers a solvency invariant check as described in Section 6.4. This linkage ensures that the scoring system not only drives optimization but also serves as an early-warning mechanism for approaching invariant violations.

6. Solvency Invariants

6.1 Definition of Solvency in Settlement OS

In the context of a settlement operating system, solvency is defined as the system's ability to honor all committed settlements across all rails at any given moment. A system is solvent if and only if, for every rail r in the set of all rails R , the total committed obligations on r do not exceed the available capacity of r . This relationship must hold at all times, including during routing updates, optimization cycles, and mesh propagation events. Formally:

$$\forall r \in R : \text{committed_obligations}(r) \leq \text{available_capacity}(r)$$

Insolvency — the violation of this condition for any rail — is classified as a safety violation in this architecture. No RSI optimization cycle, routing table update, or external ingress event may produce a system state in which this condition is violated for any rail. This constraint is enforced synchronously: no update is committed to system state until its solvency compliance has been verified.

6.2 Invariant Stack

The solvency requirement is decomposed into five distinct invariants that operate at different levels of the system hierarchy. Together, these invariants constitute the solvency invariant stack. Each invariant is defined below and summarized in the following table.

Invariant	Name	Formal Definition	Level	Enforced By
I₁	Rail Solvency	$\text{committed_obligations}(\bar{r}) \leq \text{capacity}(\bar{r})$ for all $\bar{r}$	Per-rail	JasperOS, Squirrel OS (local)
I₂	Node Solvency	$\sum \text{committed_obligations}(\bar{r}) \leq \text{node_capacity}(n)$ for all rails $\bar{r}$ attached to node n	Per-node	JasperOS, Squirrel OS (local)
I₃	Mesh Solvency	$\sum \text{committed_obligations}(\bar{r}) \leq \text{mesh_capacity}$ for all $\bar{r}$ in R	Global mesh	JasperOS (canonical)
I₄	Temporal Solvency	$\text{projected_load}(\bar{r}, t) \leq \text{capacity}(\bar{r})$ for all $\bar{r}$, for all t in $[\text{now}, \text{now} + \text{horizon}]$	Forward-looking	JasperOS (using lifecycle proof projections)
I₅	Cross-Bridge Solvency	$\text{bridge_load} \leq \min(\text{bridge_capacity}, \text{external_net})$	Boundary	Universal Bridge

Invariant	Name	Formal Definition	Level	Enforced By
		work_capacity)		

Invariants I_1 and I_2 are local invariants enforceable by individual nodes using only locally visible state. Invariant I_3 requires global aggregation and is therefore enforced only by JasperOS, which holds canonical state for all rails. Invariant I_4 is a temporal invariant that requires forward-looking analysis based on lifecycle proof projections — the committed but not yet settled obligations that are scheduled to arrive within the planning horizon. Invariant I_5 is a cross-boundary invariant enforced by the Universal Bridge.

6.3 Invariant Enforcement Mechanism

All proposed system changes that affect load distribution — including routing table updates, score-driven re-optimizations, external ingress requests, and RSI cycle outputs — are subjected to invariant checking before application. The enforcement mechanism is synchronous: no change is committed to system state until the invariant check passes. This synchronous enforcement guarantee means that the system cannot transition through an insolvent intermediate state, even transiently during an optimization cycle.

The invariant check procedure evaluates I_1 through I_5 in sequence. Failure at any invariant immediately halts the check and produces a structured rejection response containing: the invariant identifier that was violated, the specific rail or node at which the violation was detected, the magnitude of the violation (by how much committed obligations would exceed capacity), and a cause code identifying the category of the triggering change. This structured rejection is recorded in the provenance log (Section 7) and serves as input to the alternative proposal generation step.

Upon receiving a rejection, the optimization engine generates an alternative proposal — a different routing configuration that achieves a positive score delta while remaining within the feasible region defined by I_1 through I_5 . If no feasible alternative can be generated (a condition that arises when the mesh is globally near capacity), the optimization cycle is suspended and the system enters a capacity alert state.

6.4 Recovery from Near-Insolvency

When the solvency headroom score $H(r)$ for a rail r falls below the threshold $H_{\min}$, the system enters a rail preservation mode for that rail. In this mode, new load assignment to rail r is suspended; existing committed load on r is honored but no new commitments are accepted. Load that would have been routed to r is shed to the next-highest-scoring alternative rails, subject to their own invariant checks. The degraded rail r is re-scored and re-optimized at elevated priority: its scoring interval is reduced and its optimization is scheduled ahead of other pending optimization candidates.

If $H(r)$ recovers above $H_{\min}$ — as a result of completed settlements reducing committed load, or of a successful capacity expansion through optimization — rail preservation mode is exited and normal load assignment resumes. The entry, duration, and exit of rail preservation mode are all recorded in the provenance log with full lifecycle proof attestation.

7. Provenance Anchoring

7.1 Purpose of Provenance in Settlement Systems

Provenance anchoring is the mechanism by which every value transfer, routing decision, system event, and optimization cycle is permanently and unambiguously traced to its origin. In a settlement context, provenance serves a dual function: it enables auditability — any party can trace the history of any settlement or routing decision — and it enforces non-

repudiation — no node can disown an event it originated, because that event is signed and chained into the mesh-wide provenance record. Provenance anchoring is therefore not merely a logging mechanism; it is a core safety and accountability property of the architecture.

7.2 Anchoring Mechanism

Each system event E is assigned a provenance record at the moment of its occurrence. The provenance record has the following structure:

$$P(E) = \{ \textit{origin_node}, \textit{timestamp}, \textit{event_hash}, \textit{parent_hash}, \textit{signature} \}$$

The *origin_node* field identifies the node that originated the event. The *timestamp* records the moment of occurrence to sub-second precision. The *event_hash* is a cryptographic hash of the event payload, enabling verification that the event content has not been modified. The *parent_hash* is the hash of the immediately preceding provenance record in the same rail's history, forming a hash chain. The *signature* is a cryptographic signature over the entire record, generated by the originating node's private key.

The chain of provenance records for a given rail forms a directed acyclic graph (DAG) anchored at genesis records. Genesis records are the initial provenance records created when a rail is first instantiated; they have no parent and serve as the root of the chain. The DAG structure allows provenance records to branch (when a single event produces multiple downstream effects) and merge (when multiple upstream events contribute to a single downstream event), while maintaining full traceability in both directions.

7.3 Cross-System Provenance (Universal Bridge)

When a data or value payload transits the Universal Bridge — entering from an external system or exiting to an external system — the Universal Bridge generates a cross-system provenance record. This record serves to map the internal provenance chain to the provenance format used by the external

system, or vice versa. The cross-system provenance record contains: the internal provenance record identifier, the external system identifier, the external provenance format identifier, the mapping rules applied, and signatures from both the Universal Bridge and (where obtainable) the external system's gateway node.

The Universal Bridge maintains a normalization table of provenance formats for all external systems with which it is configured to interoperate. When an external system uses an unknown provenance format, the Universal Bridge rejects the ingress event and generates a format-unknown rejection record in the internal provenance log. This ensures that no external event enters the internal settlement mesh without a complete, verifiable provenance record.

7.4 Integration with RSI

Every RSI optimization cycle is itself provenance-anchored. When JasperOS generates an optimization proposal, it creates a provenance record for the proposal event before any other action is taken. This record captures what was proposed, the score delta that motivated it, the system state at the time of proposal, and the proposing node identity. When the proposal is evaluated against the invariant stack, the evaluation outcome — pass or fail — is anchored as a child record. When the optimization is propagated to the mesh, each propagation event is anchored. When a receiving node applies the optimization, its application event is anchored.

This chain of provenance records for an RSI cycle allows complete reconstruction of the cycle's history: what was optimized, why, what invariant checks were performed, what was accepted or rejected, and where in the mesh the accepted optimization was applied. Provenance anchoring of RSI cycles is mandatory — no optimization cycle may proceed past the proposal stage without generating an anchoring record.

8. Lifecycle Proofs

8.1 Definition

A lifecycle proof is a formal, cryptographically attested record that a given rail, node, or settlement has passed through a required lifecycle stage transition correctly, with all preconditions satisfied before the transition and all postconditions satisfied after. Lifecycle proofs constitute the formal verification layer of the architecture: they are machine-checkable witnesses that the system's invariants were satisfied at each stage transition, providing a stronger guarantee than provenance records alone.

The lifecycle of a rail or node proceeds through four stages: Initialization (the rail is created and configured, but not yet accepting load), Active (the rail is operational and accepting load, subject to solvency constraints), Optimization (the rail is undergoing a scoring-driven re-optimization, which may temporarily alter its configuration), and Archival or Termination (the rail has completed all committed obligations and is being decommissioned). Each transition between stages requires a lifecycle proof to be generated and verified before the transition is committed.

8.2 Proof Structure

A lifecycle proof for a rail r transitioning to stage s has the following structure:

$$LP(r, s) = \{ rail_id, stage, preconditions_hash, postconditions_hash, witness_nodes, timestamp, signature \}$$

The *rail_id* uniquely identifies the rail. The *stage* field identifies the destination stage of the transition. The *preconditions_hash* is a cryptographic hash of the set of state assertions that must hold before the transition — for example, "all committed obligations on this rail are within capacity" for the Initialization-to-Active transition. The *postconditions_hash* is a hash of the assertions that must hold after the transition. The *witness_nodes* field lists

the set of mesh nodes that have independently verified the preconditions and co-signed the proof, providing Byzantine-fault-tolerant attestation. The *timestamp* and *signature* fields complete the record.

The witness set is required to constitute a Byzantine-fault-tolerant quorum: if the mesh contains n nodes capable of witnessing, at least $\lfloor (n + 1) / 3 \rfloor$ witnesses are required for the proof to be accepted, ensuring that proof acceptance is robust against up to one-third of witnesses being faulty or malicious.

8.3 Use in RSI Cycles

Lifecycle proofs are integrated into the RSI optimization cycle at two mandatory checkpoints. Before an RSI cycle modifies a rail or routing table, it must generate a precondition lifecycle proof attesting that the preconditions for the optimization stage transition are met — specifically, that the rail is currently in an Active or Optimization stage and that all solvency invariants hold at the point of optimization initiation. This precondition proof is broadcast to the witness set for co-signature before the optimization proceeds.

After the optimization cycle completes and the new routing configuration has been applied at all affected nodes, a postcondition lifecycle proof is generated by each affected node, attesting that the postconditions hold — that the new configuration satisfies all solvency invariants and that the rail's score delta is positive. If any node finds that postconditions do not hold — for example, because the applied optimization caused a local invariant violation that was not detected during the global invariant check — the optimization is rolled back at that node, and a failure proof is generated and propagated to JasperOS for analysis. JasperOS records the failure proof in the lifecycle proof log and uses it as input to the next optimization cycle.

8.4 Lifecycle Proof Verification

Any node in the mesh can verify a lifecycle proof using only the proof record itself and the current system state, without communication with the proof originator. Verification proceeds by recomputing the hashes of the preconditions and postconditions from current state and comparing them to the hashes in the proof record, then verifying the cryptographic signatures of the originating node and all witness nodes. Verification is deterministic: the same proof will always verify to the same result given the same system state.

JasperOS maintains the canonical lifecycle proof log — an append-only, hash-chained record of all lifecycle proofs generated across the mesh. Squirrel OS nodes maintain local subsets of this log, covering the proofs relevant to their directly attached rails and their recent optimization history. The lifecycle proof log provides the evidentiary basis for external audit, regulatory compliance, and formal verification efforts.

9. Mesh-Level Propagation

9.1 Propagation Architecture

The settlement mesh is a partially connected graph composed of JasperOS root nodes, Squirrel OS leaf nodes, and Universal Bridge boundary nodes. Partial connectivity is deliberate: full connectivity (every node connected to every other node) does not scale to large mesh sizes and introduces unnecessary redundancy in propagation paths. The mesh topology is maintained by JasperOS, which tracks node membership, connection status, and propagation path availability. Topology changes — node arrivals, departures, and link failures — are propagated as control events via the control rail subsystem.

Four categories of propagation channel are defined in the architecture, each optimized for the characteristics of the data it carries:

9. **Score Channels:** Carry rail score updates from Squirrel OS nodes to JasperOS and canonical score broadcasts from JasperOS to all nodes. Score messages are compact and latency-sensitive.
10. **Routing Update Channels:** Carry versioned, signed routing table updates from JasperOS to all affected nodes. Routing updates are larger payloads and require guaranteed delivery with acknowledgment.
11. **Provenance Channels:** Carry provenance records from originating nodes to JasperOS and, where relevant, between nodes during cross-system events. Provenance messages are append-only and require strong ordering guarantees.
12. **Lifecycle Proof Channels:** Carry lifecycle proof messages between proof originators, witness nodes, and JasperOS. Lifecycle proof messages require multi-party delivery and witness acknowledgment before the proof is considered complete.

9.2 Propagation Protocol

All four propagation channel types use a common signed gossip protocol as their underlying transport. A gossip message carries: a typed payload (score update, routing update, provenance record, or lifecycle proof), the identity and cryptographic signature of the originating node, the propagation path (a list of nodes through which the message has transited, to enable loop detection), and an expiry timestamp after which the message is discarded without forwarding.

Upon receiving a gossip message, each node performs three checks before forwarding: signature validation (the originating node's signature must be cryptographically valid), invariant compliance (for routing updates, the update must pass a local invariant check), and deduplication (the message hash must not be present in the node's recent message cache). Messages that

fail any check are discarded and a rejection record is generated. Messages that pass all checks are applied to local state and forwarded to all neighboring nodes not already in the propagation path.

9.3 RSI Propagation

When JasperOS generates an accepted optimization — a new routing table version or updated rail weight configuration that has passed the full invariant stack — it packages the optimization as a signed RSI update message. The RSI update message contains the new routing configuration, the provenance record of the optimization cycle that produced it, the pre- and post-condition hashes from the associated lifecycle proofs, and a priority flag indicating whether the update should be propagated at standard or elevated priority.

RSI update messages are propagated to all Squirrel OS nodes within the subgraph affected by the optimization. The affected subgraph is determined by JasperOS based on which rails and nodes are covered by the updated routing configuration. Each receiving Squirrel OS node applies the update only after its local invariant check passes. Nodes for which the local invariant check fails — indicating a local condition that was not visible to JasperOS during the global invariant check — send a signed rejection message back to JasperOS via the control rail. JasperOS collects rejections, analyzes the cause codes, generates a revised optimization proposal that accounts for the locally-reported constraint, and re-propagates. This iteration continues until either the optimization is accepted by all affected nodes or JasperOS determines that no feasible optimization exists under current conditions.

9.4 Convergence Properties

The propagation protocol can be shown informally to converge to a consistent state across all non-faulty nodes, under the assumption that the mesh remains connected (or eventually reconnects after a partition). The argument proceeds as follows. Each optimization cycle produces a single canonical routing table version, signed by JasperOS. This version is propagated to all

nodes via the gossip protocol. Because the gossip protocol guarantees eventual delivery to all reachable nodes (under the assumption of eventual connectivity and message non-expiry before delivery), all non-faulty nodes eventually receive the canonical version. Because invariant checks are deterministic, all non-faulty nodes that hold the same system state will either all accept or all reject the update. In the acceptance case, all nodes converge to the same routing table version. In the rejection case, all nodes retain the previous routing table version and JasperOS generates a new proposal, repeating the process.

Convergence time is bounded above by the mesh diameter (the maximum hop count between any two nodes) multiplied by the per-hop propagation latency. Solvency invariants are preserved throughout the propagation process: because each node checks invariants before applying any update, no intermediate state — at any node, at any point in the propagation — can be insolvent. The system is therefore solvent at every moment, not merely in converged steady state.

10. The RSI Feedback Loop: Unified Description

10.1 The Five-Layer Stack

The RSI architecture is organized as a five-layer stack in which each layer provides a distinct function and the layers interact to produce a system that is simultaneously self-improving and formally bounded. The five layers are:

Layer	Component	Function in RSI	Safety Role
1. Rail Scoring	Scoring Engine (JasperOS / Squirrel OS)	Produces efficiency signals that identify optimization	Drives improvement; provides early warning of solvency

Layer	Component	Function in RSI	Safety Role
		candidates	degradation
2. Solvency Invariants	Invariant Checker (JasperOS / Squirrel OS / Universal Bridge)	Constrains the feasible region of permissible improvements	Prevents unsafe optimization; enforces hard safety boundaries
3. Provenance Anchoring	Provenance Engine (all nodes)	Records every self-modification with full lineage and cryptographic integrity	Enables full audit; supports non-repudiation of optimization events
4. Lifecycle Proofs	Proof Engine (JasperOS / witness nodes)	Formally attests pre/post conditions of each optimization cycle	Provides machine-checkable safety evidence; enables rollback on postcondition failure
5. Mesh Propagation	Propagation Protocol (all nodes)	Distributes accepted optimizations across the system	Ensures all nodes converge to same safe state; preserves invariants during propagation

10.2 Cycle Operation

A complete RSI cycle proceeds through the following steps:

13. **Local Score Computation:** Squirrel OS nodes compute local rail scores using the scoring function $S(r) = w_1 \cdot T(r) + w_2 \cdot L(r) + w_3 \cdot H(r) + w_4 \cdot P(r)$ and propagate signed score messages to JasperOS via score channels.

14. **Score Aggregation and Candidate Identification:** JasperOS aggregates received scores into canonical scores, computes score deltas, and identifies rails or routing configurations for which the delta exceeds the optimization threshold $\Delta_{\min}$.
15. **Optimization Proposal Generation:** JasperOS generates a proposed optimization — a new routing table version or updated rail weight configuration — that is predicted to increase the aggregate score across identified candidates.
16. **Invariant Check:** The proposed optimization is checked against I_1 through I_5 . If any invariant is violated, a structured rejection is recorded in the provenance log and an alternative proposal is generated (returning to step c).
17. **Precondition Lifecycle Proof:** If the invariant check passes, JasperOS generates a precondition lifecycle proof, broadcasts it to the witness set for co-signature, and records the signed proof in the lifecycle proof log.
18. **Provenance Anchoring:** The accepted optimization and its precondition proof are anchored in the provenance DAG, generating signed provenance records for the proposal and acceptance events.
19. **Mesh Propagation:** The signed RSI update message is propagated to all affected Squirrel OS nodes via the routing update channels.
20. **Local Application:** Each receiving Squirrel OS node performs a local invariant check. If the local check passes, the node applies the optimization and generates a local application provenance record. If the local check fails, the node sends a rejection message to JasperOS.
21. **Postcondition Lifecycle Proofs:** Each node that successfully applies the optimization generates a postcondition lifecycle proof, attesting

that postconditions hold locally. These proofs are forwarded to JasperOS for aggregation into the canonical lifecycle proof log.

22. Cycle Completion: Once JasperOS receives postcondition proofs from all affected nodes (or processes rejections from non-compliant nodes and generates revised proposals), the cycle is marked complete. Scores are recomputed and the next cycle is scheduled.

10.3 Self-Improvement Guarantee

The system satisfies a monotonic improvement property in expected aggregate rail score: each completed RSI optimization cycle produces a routing configuration with a higher expected aggregate score than the configuration it replaces, within the invariant-bounded feasible region. This property follows from the definition of the optimization: JasperOS generates proposals only when the predicted score delta is positive (exceeds $\Delta_{\min}$), and only proposals that pass the full invariant check are applied. The feasible region defined by I_1 through I_5 acts as a lower bound on system state — the system cannot improve its way into an insolvent configuration, and it cannot deteriorate below the invariant floor without triggering rail preservation mode.

This is the formal sense in which the system is "recursively self-improving": each optimization cycle uses the outputs of previous cycles — the current score state, the current routing configuration, the historical provenance log — as inputs to generate a structurally better configuration. The improvement is recursive because each better configuration enables more accurate scoring, which enables better subsequent optimizations. The improvement is bounded because every step is constrained by the invariant stack.

10.4 Relationship to AI Safety Principles

The RSI architecture described in this paper instantiates three foundational principles from the AI safety literature by formal construction rather than by heuristic or policy:

Corrigibility: The system is corrigible in the sense that any optimization cycle can be halted by an invariant violation. An operator or external authority can induce halting by adjusting invariant thresholds or by withdrawing witness node co-signatures from lifecycle proofs. The system cannot optimize its way past a corrigibility constraint because invariant checks are synchronous and mandatory.

Containment: No RSI cycle escapes the invariant stack. The stack is not a soft constraint or a penalty term in an objective function; it is a hard gate through which every proposed change must pass. A system change that violates any invariant is categorically rejected, not penalized or probabilistically discouraged. This provides a containment guarantee that is stronger than any machine-learning-based safety approach.

Auditability: Full provenance of every self-modification is maintained in the provenance DAG and lifecycle proof log. Any external auditor — a regulator, a patent reviewer, a safety researcher — can reconstruct the complete history of the system's self-optimization from these records. The records are cryptographically signed and hash-chained, preventing post-hoc modification.

Classical unconstrained RSI proposals — Gödel Machines, AIXI-style agents — do not satisfy containment or auditability by construction; these properties must be added as external constraints or are absent altogether. The architecture described here achieves all three properties as intrinsic structural features of the RSI loop itself.

11. Patent Coverage and Intellectual Property

11.1 Overview of Patented Methods

The routing, scoring, and solvency enforcement methods described in this paper are covered by patent applications filed by the author (Leon, PheonixSecure). The specific elements for which patent protection has been sought include: the deterministic rail scoring function and its multi-objective weighting framework; the solvency invariant stack (I_1 through I_5) and the synchronous enforcement mechanism; the cross-system provenance normalization methodology implemented in the Universal Bridge; and the lifecycle proof generation, witness attestation, and verification protocol. Readers and practitioners seeking to implement any of these methods in commercial or operational systems should consult the relevant patent filings and obtain appropriate licensing from the patent holder before proceeding.

11.2 Open Description for Research Purposes

This paper is published as an open-access preprint on the Zenodo research repository under the Creative Commons Attribution 4.0 International (CC BY 4.0) license. The description of the architecture contained herein is provided for scientific and scholarly purposes: to enable peer review, academic analysis, formal verification research, and citation. The CC BY 4.0 license grants rights to share and adapt this descriptive content with attribution; it does not grant rights to implement the described patented methods without a separate commercial or research license from the patent holder. Inquiries regarding licensing should be directed to the author.

Note to Patent Reviewers

This document is submitted in conjunction with, and as a scholarly companion to, pending patent applications covering the methods

described herein. The formal definitions, invariant formulations, and component descriptions in this paper are consistent with and supplementary to the claims language in those applications. This document does not constitute a waiver of any patent rights.

12. Conclusion

12.1 Summary

This paper has presented a formal architectural description of recursive self-improvement as instantiated in the JasperOS, Squirrel OS, and Universal Bridge settlement operating system family. The central contribution is the demonstration that RSI — typically associated with unconstrained and potentially unsafe AI self-modification — can be realized as a formally bounded, invariant-preserving optimization loop applicable to distributed economic settlement systems.

The architecture achieves this through a five-layer stack in which rail scoring produces continuous efficiency signals, solvency invariants constrain the feasible region of permissible improvements, provenance anchoring records every self-modification with cryptographic integrity, lifecycle proofs formally attest the correctness of each optimization cycle, and mesh propagation distributes accepted optimizations across the system while preserving invariants throughout. The three system components — JasperOS (root authority), Squirrel OS (edge leaf nodes), and Universal Bridge (boundary interoperability layer) — each instantiate these layers at their respective tiers of the deployment hierarchy.

The system satisfies a monotonic improvement property in aggregate rail score, bounded below by the solvency invariant floor, and satisfies the AI

safety properties of corrigibility, containment, and auditability by formal construction. No optimization cycle can produce an insolvent, unprovable, or unprovenanced system state.

12.2 Future Work

Several directions for future work are identified:

23. **TLA+ Formal Specification:** A complete TLA+ model of the five-layer invariant stack would enable automated model checking of the convergence, solvency preservation, and liveness properties described informally in this paper. This is the most immediate priority for formal verification of the architecture.
24. **Extension to Heterogeneous External Networks:** The Universal Bridge currently supports a defined set of external system types. Extension to fully arbitrary external networks — including legacy financial messaging systems and emerging decentralized protocol stacks — requires a generalized provenance normalization framework beyond the current mapping table approach.
25. **Convergence Benchmarking at Scale:** The informal convergence bound of mesh diameter multiplied by propagation latency should be empirically validated across mesh configurations of varying size, topology density, and node failure rates. Simulation and testbed deployment are both contemplated.
26. **Zero-Knowledge Proof Integration:** The current provenance anchoring mechanism provides transparency to all nodes in the mesh. In contexts where provenance privacy is required — for example, when competing commercial parties share a settlement mesh — zero-knowledge proof systems could provide attestation of invariant compliance without revealing the underlying provenance data.

Integration of ZK proof systems with the lifecycle proof mechanism is a promising direction.

27. Weight Optimization for the Scoring Function: The weights w_1 through w_4 in the rail scoring function are currently set by system operators. An RSI-driven meta-optimization of these weights — subject to the constraint that $w_3 \geq w_{3\min}$ — would further close the self-improvement loop and reduce the need for manual parameter tuning.

References

- [1] Schmidhuber, J. (2003). *Gödel Machines: Self-Referential Universal Problem Solvers Making Provably Optimal Self-Improvements*. Technical Report IDSIA-19-03, IDSIA, Lugano, Switzerland. arXiv:cs/0309048.
- [2] Lamport, L., Shostak, R., & Pease, M. (1982). The Byzantine Generals Problem. *ACM Transactions on Programming Languages and Systems (TOPLAS)*, 4(3), 382-401. <https://doi.org/10.1145/357172.357176>
- [3] Holzmann, G. J. (1997). The Model Checker SPIN. *IEEE Transactions on Software Engineering*, 23(5), 279-295. <https://doi.org/10.1109/32.588521>
- [4] Nakamoto, S. (2008). *Bitcoin: A Peer-to-Peer Electronic Cash System*. Self-published white paper. <https://bitcoin.org/bitcoin.pdf>
- [5] Lamport, L. (2002). *Specifying Systems: The TLA+ Language and Tools for Hardware and Software Engineers*. Addison-Wesley. ISBN 0-321-14306-X.
- [6] Hutter, M. (2005). *Universal Artificial Intelligence: Sequential Decisions Based on Algorithmic Probability*. Springer. <https://doi.org/10.1007/b138233>
- [7] Demers, A., Greene, D., Hauser, C., Irish, W., Larson, J., Shenker, S., Sturgis, H., Swinehart, D., & Terry, D. (1987). Epidemic algorithms for replicated database

maintenance. *Proceedings of the Sixth Annual ACM Symposium on Principles of Distributed Computing (PODC)*, 1-12.

- [8] Castro, M., & Liskov, B. (1999). Practical Byzantine fault tolerance. *Proceedings of the Third USENIX Symposium on Operating Systems Design and Implementation (OSDI)*, 173-186.

Repository Note: This document was prepared for submission to Zenodo (zenodo.org), the open-access research repository operated by CERN. A DOI will be assigned by Zenodo upon upload and will replace the placeholder in the title page and reference [10] above. The assigned DOI should be used in all citations of this work following DOI assignment.